

ADVANCED

overall tree
= binary tree of mini trees

mini trees

micro trees

actual nodes

$\frac{1}{4} \lg n$ nodes

$\lg n$ nodes



5

Integer Data Structures

Advanced Data Structures · Summer 2026

Prof. Dr. Sebastian Wild

5 Integer Data Structures

- 5.1 Integer Dictionaries
- 5.2 Perfect Hashing
- 5.3 Dynamic Perfect Hashing
- 5.4 Binary Tries
- 5.5 x-Fast Tries
- 5.6 y-Fast Tries
- 5.7 Lower Bounds



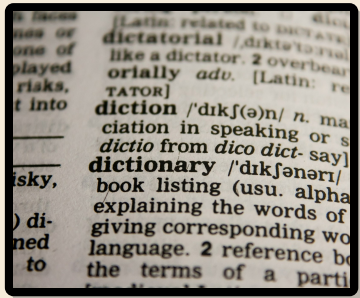
5.1 Integer Dictionaries

Recall: Symbol table ADT

Java: `java.util.Map<K,V>`, C++: `std::(unordered_)map`

Symbol table / Dictionary / Map / Associative array / key-value store:

Python `dict {k:v}`



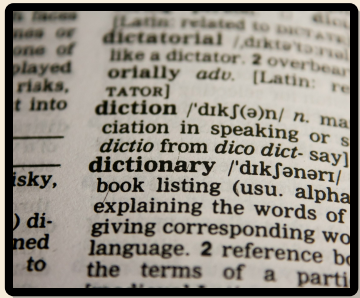
- ▶ `put(k, v)` Python dict: `d[k] = v`
Put key-value pair (*k*, *v*) into table
- ▶ `get(k)` Python dict: `d[k]`
Return value associated with key *k*
- ▶ `delete(k)` Python dict: `del d[k]`
Remove key *k* (any associated value) from table
- ▶ `contains(k)` Python dict: `k in d`
Returns whether the table has a value for key *k*
- ▶ `isEmpty(), size(), create()`

Recall: Symbol table ADT

Java: `java.util.Map<K,V>`, C++: `std::(unordered_)map`

Symbol table / Dictionary / Map / Associative array / key-value store:

Python `dict {k:v}`



- ▶ `put(k, v)` Python dict: `d[k] = v`
Put key-value pair (*k*, *v*) into table
- ▶ `get(k)` Python dict: `d[k]`
Return value associated with key *k*
- ▶ `delete(k)` Python dict: `del d[k]`
Remove key *k* (any associated value) from table
- ▶ `contains(k)` Python dict: `k in d`
Returns whether the table has a value for key *k*
- ▶ `isEmpty(), size(), create()`

Unlike before: Focus here on unordered case

- ▶ no rank-based queries
- ▶ values uninteresting
- ⇒ summarize get and put as `lookup(k)`

Integer Dictionaries

For unsorted dictionaries, best solutions based on *hashing*.

- ▶ Whatever the actual objects we store, always apply a hash function first

- ↪ Consider here directly (dynamic) sets of integers

Hash Tables

Notation

- ▶ **universe** $U = [0..u)$ of allowed values
- ▶ **universe size** $u = |U|$, usually $u = 2^w$
- ▶ $S \subseteq U$ dynamic set of integers stored in our dictionary
- ▶ $n = |S|$ current **size** of set, $S = \{x_1, \dots, x_n\}$

Hash Tables

Notation

- ▶ **universe** $U = [0..u)$ of allowed values
- ▶ **universe size** $u = |U|$, usually $u = 2^w$
- ▶ $S \subseteq U$ dynamic **set** of integers stored in our dictionary
- ▶ $n = |S|$ current **size** of set, $S = \{x_1, \dots, x_n\}$
- ▶ use **hash table** $T[0..m)$ with m buckets
- ▶ **hash function** $h : U \rightarrow [0..m)$
- ↪ element $x \in U$ stored in $T[h(x)]$

Hash Tables

Notation

- ▶ **universe** $U = [0..u)$ of allowed values
- ▶ **universe size** $u = |U|$, usually $u = 2^w$
- ▶ $S \subseteq U$ dynamic **set** of integers stored in our dictionary
- ▶ $n = |S|$ current **size** of set, $S = \{x_1, \dots, x_n\}$
- ▶ use **hash table** $T[0..m)$ with m **buckets**
- ▶ **hash function** $h : U \rightarrow [0..m)$
- ↪ element $x \in U$ stored in $T[h(x)]$
- ▶ in general: $T[h(x)]$ may be a **secondary data structure**
 - ▶ linked list ↪ *chaining hashing*
 - ▶ sequence of positions in T ↪ *open-addressing hashing*

Hash Tables

Notation

- ▶ **universe** $U = [0..u)$ of allowed values
- ▶ **universe size** $u = |U|$, usually $u = 2^w$
- ▶ $S \subseteq U$ dynamic **set** of integers stored in our dictionary
- ▶ $n = |S|$ current **size** of set, $S = \{x_1, \dots, x_n\}$
- ▶ use **hash table** $T[0..m)$ with m **buckets**
- ▶ **hash function** $h : U \rightarrow [0..m)$
- ↪ element $x \in U$ stored in $T[h(x)]$
- ▶ in general: $T[h(x)]$ may be a **secondary data structure**
 - ▶ linked list ↪ *chaining hashing*
 - ▶ sequence of positions in T ↪ *open-addressing hashing*
- ▶ if $h(x) = h(y)$, we have a **collision** (between x and y)

Hashing must be randomized

Collisions are inevitable

- ▶ usually want S represented with $O(n)$ space

$\rightsquigarrow m = O(n)$

- ▶ typically thus $|U| = 2^w \gg m$

$\rightsquigarrow h$ must have many collisions

$\rightsquigarrow$ For any *fixed* hash function h , worst-case set S has many collisions

$\rightsquigarrow$ cannot have meaningful worst-case time bounds (even if typical case good)

365 $\approx$ 25 people to get 50%
collision for birthdays

$$n \approx 2\sqrt{m}$$

Hashing must be randomized

Collisions are inevitable

- ▶ usually want S represented with $O(n)$ space

$\rightsquigarrow m = O(n)$

- ▶ typically thus $|U| = 2^w \gg m$

$\rightsquigarrow h$ must have many collisions

$\rightsquigarrow$ For any *fixed* hash function h , worst-case set S has many collisions

$\rightsquigarrow$ cannot have meaningful worst-case time bounds (even if typical case good)

Randomized Hashing

- ▶ draw *random* $h \in \mathcal{H}$

$\rightsquigarrow$ running time expected over random choice, worst-case w.r.t. S

- ▶ allowing all $\mathcal{H} = \{h \mid h : U \rightarrow [0..m)\}$ requires m^u space ⚡
 $\log(m^u) = u \cdot \log m$

$\rightsquigarrow$ must choose sufficiently small $\mathcal{H}$

Universal Hashing

Many guarantees of varying strength studied for $\mathcal{H}$... here only simplest needed

Definition 5.1 (Universal Family)

Let $\mathcal{H}$ be a set of hash functions from U to $[m]$ and $|U| \geq m$.

Assume $h \in \mathcal{H}$ is chosen uniformly at random.

(a) Then $\mathcal{H}$ is called a *c-universal* if

$$\forall x_1, x_2 \in U : x_1 \neq x_2 \implies \mathbb{P}[h(x_1) = h(x_2)] \leq \frac{c}{m}.$$

(b) $\mathcal{H}$ is called *strongly universal* or *pairwise independent* if

$$\forall x_1, x_2 \in U, y_1, y_2 \in R : x_1 \neq x_2 \implies \mathbb{P}[h(x_1) = y_1 \wedge h(x_2) = y_2] \leq \frac{1}{m^2}.$$

for fully random h

$$h(x_1) \stackrel{\mathcal{D}}{=} \mathcal{U}([m])$$

$$h(x_2) \stackrel{\mathcal{D}}{=} \mathcal{U}([m])$$

$$\implies \mathbb{P}[h(x_1) = h(x_2)] = \frac{1}{m}$$

Examples of universal families

Universal families

$$\blacktriangleright \mathcal{H}_1 = \{h_{ab} : a \in [1..p), b \in [0..p)\} \quad (c = 2)$$

$$\blacktriangleright \mathcal{H}_0 = \{h_{ab} : a \in [0..p), b \in [0..p)\} \quad (c = 4)$$

$$\blacktriangleright \mathcal{H}_2 = \{h_a : a \in [1..2^k), a \text{ odd}\} \quad (c = 2)$$

$$h_{ab}(x) = (a \cdot x + b \bmod p) \bmod m \quad p \text{ prime}, p \geq m$$

$$h_a(x) = (a \cdot x \bmod 2^k) \operatorname{div} 2^{k-\ell} \quad u = 2^k, m = 2^\ell$$

What does universality buy us?

Lemma 5.2 (Universal collisions)

Consider hashing n keys $x_1, \dots, x_n$ into m buckets using a random hash function h chosen from a c -universal family of hash functions $\mathcal{H}$. Let C denote the number of pairwise collisions, i. e., $C = |\{(i, j) : 1 \leq i < j \leq n \wedge h(x_i) = h(x_j)\}|$.

Then $\mathbb{E}[C] < \frac{cn^2}{2m}$.

$$C \leq \binom{n}{2} \quad C \geq \binom{\frac{n}{m}}{2} \cdot m \approx \frac{n^2}{m} \cdot \frac{1}{2}$$

Lemma 5.3 (Universal bin size)

Consider the scenario of Lemma 5.2. Let $X_j = |T[j]|$ be the number of elements in bucket j .

(a) $\mathbb{E}[X_h(x_i)] \leq 1 + c \cdot \frac{n}{m}$

(b) Then $\mathbb{P}\left[\max X_j > \sqrt{2c} \cdot \frac{n}{\sqrt{m}}\right] \leq \frac{1}{2}$